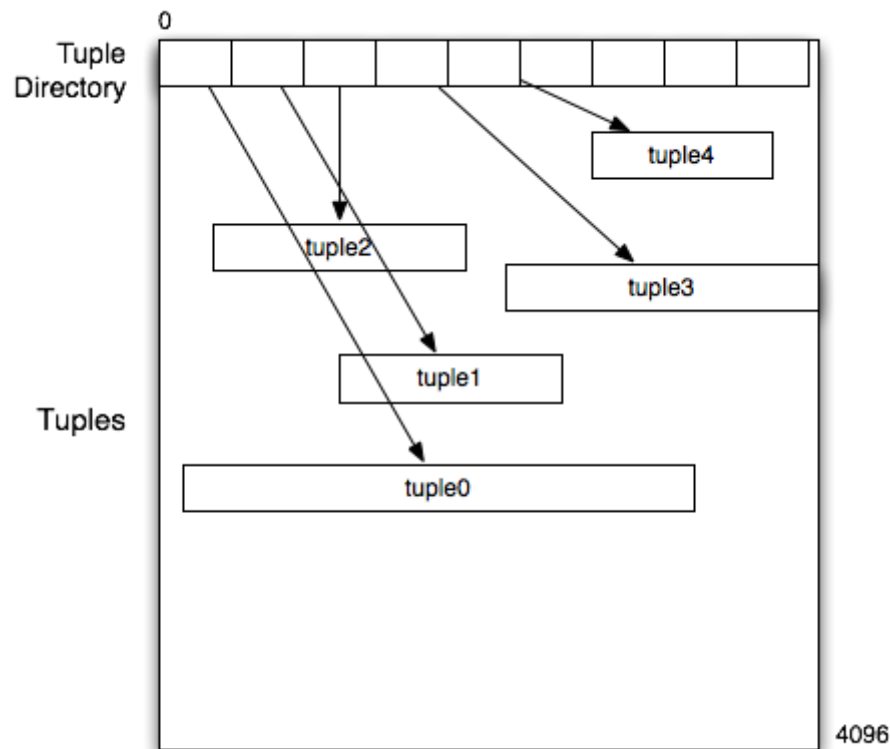


[\[Show with no answers\]](#) [\[Show with all answers\]](#)

1. Assume that a data file is composed of 4KB pages, where each page is structured as follows:



The start of the page contains a tuple directory which is a sequence of three-byte values, where the first 12 bits contain the offset into the page of the tuple and the second 12 bits contain the tuple length.

Write a C function that takes three parameters: an open file descriptor, a page number and a record number and reads the data for the corresponding record. Do *not* read the whole page; read just enough of the data to solve the problem. Dynamically allocate a memory buffer large enough to hold the tuple data when read in. The function should return a pointer to the start of the tuple memory buffer.

The function should do appropriate error-checking and return `NULL` in the case that any operation cannot be completed. Use the following function template:

```
char *getTuple(int inFile, int pageNumber, int recNumber) { ... }
```

Hint: when a directory value is read into memory, the high-order bits contain the offset and the low-order bits contain the length.



Use only the low-level i/o operators (system calls) such as `open()`, `read()`, `write()`, `lseek()`, etc.

[\[show answer\]](#)

2. Consider a data file containing tuples with a page structure similar to that in the previous question. Pages are 4KB in size, and each page contains a tuple directory with 100 entries in it, where each entry is 3-bytes long. Assuming that the (minimum,average,maximum) tuple lengths are (32,64,256) bytes and that the file has 100 pages, determine the following:

- The minimum number of tuples that the file can hold
- The maximum number of tuples that the file can hold

[\[show answer\]](#)

3. Consider a variation on the above scenario. Rather than pages having a fixed size tuple directory, the tuple directory can grow and shrink depending on the number of tuples in the page. For this to work, the tuple directory starts at the bottom of the page (address 0) and grows up, while tuples are added from the top of the page (address 4095) and grow down. If all other factors are the same (total 100 pages, (min,avg,max) tuple lengths (32,64,128)), what is the maximum number of tuples that the file can hold? You may assume that tuples can begin at any address (i.e. they do not have to start at a 4-byte address).

[\[show answer\]](#)

4. [Based on Garcia-Molina/Ullman/Widom 13.7.1]
Consider the following relational table:

```
create table Patient (  
    id          integer primary key,  
    ssn         char(20),  
    name        varchar(30),  
    addr        varchar(60),  
    history     varchar(200),  
    born        date  
);
```

Consider how records of this type might be represented when stored in disk blocks. If a pointer within a record requires 4 bytes and the record length is also stored as a 4-byte quantity, how many bytes would be needed for each `Patient` record, excluding of the space required to store the variable-length fields?



For variable-length records (**varchar**), assume that we don't need a terminating char (e.g. **'\0'**) but we do need to allocate a multiple of 4 bytes to the field to ensure alignment of possible following integer fields. (These additional bytes would not be necessary if the fixed-length fields were all stored at the front of the record.)

How many bytes would be required for the following instances of **Patient** records:

```
insert into Patient values (
    12345678, '222-444-555-7777',
    'John Smith', (10)
    '50 Smith St, Smithton, 2345', (28)
    'Previously diagnosed with URTI, also weak ankles', (48)
    '1966-12-2'
);
insert into Patient values (
    87654321, '123-456-654-4321',
    'Jane Brown', (10)
    '15 Brown St, Brownsville, 2427', (31)
    'Prior history of urinary tract infections', (41)
    '1966-12-2'
);
```

(Note that the (string lengths) given after each string are not part of the **insert** statement).

[\[show answer\]](#)

5. PostgreSQL tuples have an array of flags to indicate where NULL values occur in tuples. Two of the critical functions for manipulating tuples in PostgreSQL (**heap_form_tuple()** and **heap_modify_tuple()**) accept a parameter which is an array of flags for NULLs, as well as accepting an array of **Datum** values. This array contains only the non-NULL attribute values.

For example, a tuple like $R(42, null, 'abc', -5)$ would be represented by the two arrays: flags=(0,1,0,0) and values=(42,'abc',-5).

Why doesn't PostgreSQL simply include the NULL values in the array of **Datum** values?

[\[show answer\]](#)

6. Consider a **Students** relation defined as:

```
CREATE TABLE Students (
    id#    integer,      name varchar(30),
    gender varchar(1),   birth date,
    degree varchar(10),  score real
```



```
);
```

Assume that:

- There are 20,000 student records
- The relation is stored in one file
- The file is composed of 1024-byte blocks
- Average block-fetch time (T_r) is 10ms
- A **date** is represented by an 8-byte **long** value
- All numeric values must be stored on 4-byte address boundaries
- Average length of **name** value is 15 chars
- Average length of **degree** value is 5 chars

Consider two possible storage structures:

- i. fixed-length records with a presence bit-vector
- ii. variable-length records with a fixed-size directory containing one-byte offset values

For each of these structures:

- a. Show the internal record structure and compute the (average) size of a record
- b. Compute how many blocks are needed to store the whole relation
- c. Compute how long it takes to answer a query on **id#** if the file is sorted on this field (worst case value)

[\[show answer\]](#)

